

Count Sub Islands (LC 1905)

LeetCode Problem Analysis & Solutions

Zepto, Amazon

© Authored by Arpit Raj, LNMIIT Jaipur, July 22, 2026

Visual Example

G1					G2				
1	1	1	0	0	1	1	1	0	0
0	1	1	1	1	0	0	1	1	1
0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	1	0	1	1	0
1	1	0	1	1	0	1	0	1	0

Thence ans = 3

Concepts

- while doing DFS / BFS on island 2 we can maintain a flag for each island (i.e for each queue level or for each recursion level).
- if **1 on G2 but 0 on G1** then it cant be a subisland so flag = false

Code Implementations

DFS Solution

```
class Solution {
public:
    vector<pair<int, int>> dir{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    int n;
    int m;
```

```

    void dfs(int r, int c, vector<vector<int>>& grid1, vector<vector<int>>& grid2,
bool& flag) {
        grid2[r][c] = 0;
        if (grid1[r][c] == 0) flag = false;

        for (auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;

            if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                if (grid2[nr][nc] == 1) {
                    dfs(nr, nc, grid1, grid2, flag);
                }
            }
        }
    }

int countSubIslands(vector<vector<int>>& grid1, vector<vector<int>>& grid2) {
    n = grid1.size();
    m = grid1[0].size();
    int ans = 0;
    bool flag;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid2[i][j] == 1) {
                flag = true;
                dfs(i, j, grid1, grid2, flag);
                if (flag) ans++;
            }
        }
    }
    return ans;
}
};

```

BFS Solution

```

class Solution {
public:
    int countSubIslands(vector<vector<int>>& grid1, vector<vector<int>>& grid2) {
        int n = grid2.size();
        int m = grid2[0].size();

        vector<pair<int, int>> dir = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
        queue<pair<int, int>> q;

        int ans = 0;
        bool flag;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (grid2[i][j] == 1) {
                    flag = true;
                    grid2[i][j] = 0;
                    q.push({i, j});

                    while (!q.empty()) {
                        int r = q.front().first;
                        int c = q.front().second;
                        q.pop();

                        if (grid1[r][c] == 0) {
                            flag = false;
                        }

                        for (auto it : dir) {
                            int nr = r + it.first;
                            int nc = c + it.second;

                            if (nr >= 0 && nr < n && nc >= 0 && nc < m) {
                                if (grid2[nr][nc] == 1) {
                                    grid2[nr][nc] = 0;
                                    q.push({nr, nc});
                                }
                            }
                        }
                    }

                    if (flag) {
                        ans++;
                    }
                }
            }
        }

        return ans;
    }
};

```

```
}  
};
```